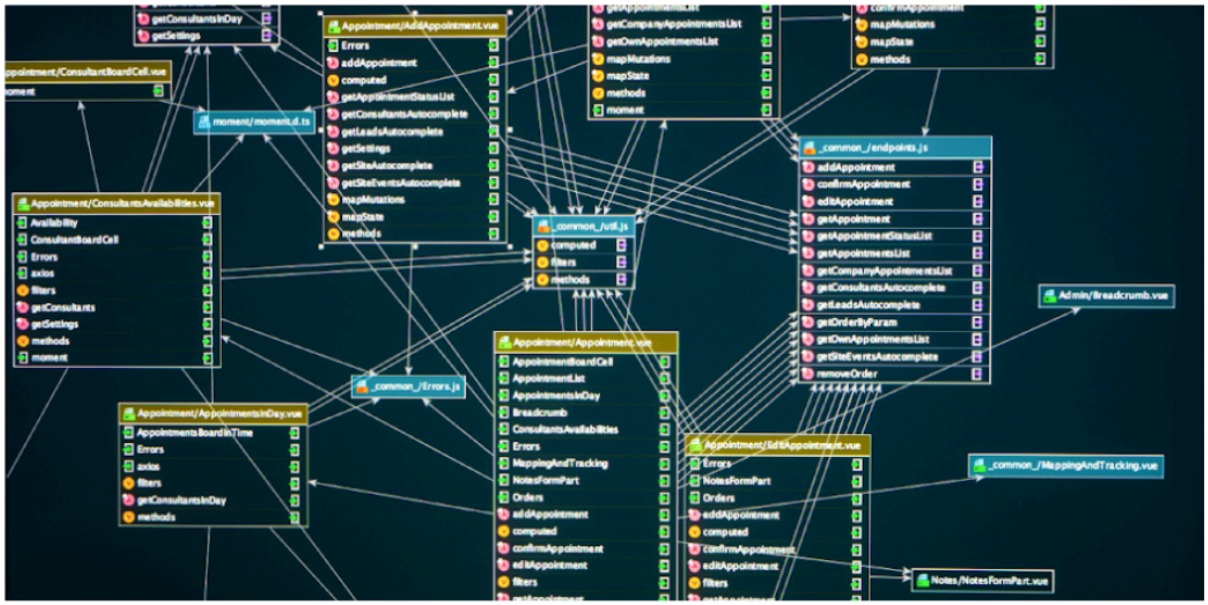
TI3093 – PRAKTIKUM PERANCANGAN SISTEM TERINTEGRASI III

SUPLEMEN PEMODELAN DATA

MODUL 5



Program Studi Teknik Industri
Fakultas Teknik Industri
Institut Teknologi Bandung
2025

DAFTAR ISI

DAFTAR ISI **1**
TERMINOLOGI DASAR **2**
I. PENDAHULUAN **3**
1.1 Tahapan Pemodelan Data 4
1.2 Entity Relationship Diagram (ERD) 6
1.3 Normalisasi 14
II. STUDI KASUS PEMODELAN DATA **18**
2.1 ERD 18
2.2 LDM 20
2.3 PDM 21
REFERENSI **22**

TERMINOLOGI DASAR

- **Data**
  Data adalah sekumpulan fakta dan statistik yang belum diolah sehingga kurang bermakna. Data-data ini perlu diorganisasikan secara logis agar dapat digunakan oleh *stakeholder*. Unit data terkecil yang dikenali oleh komputer terdiri dari karakter seperti huruf (A-Z), angka (0-9), dan simbol (?, >, :).
- ***Field* (Atribut)**
  Field adalah unit terkecil dari data yang bermakna dan disimpan dalam sebuah *file* atau basis data.
  Field dapat berupa kumpulan karakter atau alfabet. Contoh: Nama Mahasiswa, IPK, dan NIM.
- ***Record***
  Record adalah sekumpulan *field* yang terhubung secara logis dan digunakan untuk mendeskripsikan seseorang, sebuah tempat, atau sesuatu yang lain. Contoh: *record* mahasiswa yang berisikan nama, NIM, alamat, nomor telepon, tanggal kelahiran, dan IPK.
- ***File***
  File adalah sekumpulan *record* yang saling terkait. Contoh: sebuah *file* terdiri dari *record* semua mahasiswa yang saat ini belajar di Teknik Industri ITB.
- ***Database***
  Database (basis data) adalah sekumpulan *file* yang memiliki relasi (secara logika) dan didesain untuk memenuhi kebutuhan beberapa pengguna di sebuah organisasi. Contoh: *Database* mahasiswa ITB terdiri dari semua mahasiswa ITB dari berbagai program studi.
- ***Database Management System* (DBMS)**
  DBMS adalah perangkat lunak khusus yang digunakan untuk membuat, mengakses, mengontrol, dan mengelola sebuah *database*. Contoh: Microsoft SQL Server, MySQL, dan Microsoft Access.

I. PENDAHULUAN

Dalam modul 5 telah dibahas mengenai salah satu metode pengembangan sistem informasi yaitu siklus SDLC (*System Development Life Cycle*). Pada SDLC, terdapat tahap analisis kebutuhan dan desain yang terdiri dari perancangan pemodelan proses dan data. Jika pada modul sebelumnya telah dibahas mengenai pemodelan proses, pada modul ini akan dibahas lebih lanjut terkait pemodelan data.

Pemodelan data merupakan teknik yang digunakan untuk mengorganisasikan dan mendokumentasikan data-data dari suatu sistem. Model data dibuat dalam bentuk grafik yang menggambarkan struktur data dalam sistem nyata beserta karakteristik, relasi, batasan-batasan, serta transformasi data. Model ini akan digunakan antara lain sebagai fasilitas komunikasi/interaksi antara *system designer*, *application programmer*, dan *end users*.

1.1 Tahapan Pemodelan Data

Dalam melakukan pemodelan data, terdapat beberapa tahapan yang harus dilakukan agar model data dapat dibuat dengan baik dan benar. Adapun tahapan tersebut dapat dilihat sebagai berikut.

Tabel 1 Tahapan Pemodelan Data

Tahapan	Langkah	Implementasi
Perancangan Model Konseptual	**1. Analisis data dan kebutuhan** Langkah ini dibutuhkan untuk menentukan: ● Informasi yang dibutuhkan, informasi yang diharapkan akan dihasilkan sistem, dan informasi apa yang tersedia di sistem eksisting. ● Pengguna informasi, yang mencakup siapa yang menggunakan informasi dan bagaimana informasi tersebut akan digunakan oleh pengguna informasi. ● Sumber informasi, yang mencakup di mana dan bagaimana informasi tersebut diambil. ● Data yang dibutuhkan untuk menghasilkan informasi, atribut yang dimiliki data, dan relasi yang dimiliki antar data. **2. Pemodelan *entity relationship* dan normalisasi** **3. Verifikasi model data** Verifikasi model data dibutuhkan untuk memastikan sistem dapat memenuhi seluruh kebutuhan sistem. Model data harus bersifat kohesif, yang berarti masing-masing entitas memiliki hubungan yang kuat, dan model harus lengkap dan komprehensif. **4. Perancangan basis data** *Software* DBMS dipilih dengan mempertimbangkan biaya, fitur *software* DBMS, *portability*, dan *hardware* yang dibutuhkan DBMS terkait.	Pada dasarnya, terdapat beberapa cara untuk memodelkan data secara konseptual seperti dalam bentuk *Hierarchical Database Model* dan *Relational Database Model*. Pada modul ini, akan digunakan *Relational Database Model* dengan realisasi pembuatan **ERD *(Entity Relationship Diagram)*** ● Merepresentasikan objek di dunia nyata secara realistis. ● Menunjukkan data yang akan disimpan dan relasi antar data. ● Independen dengan *software* maupun *hardware* sebagai representasi model konseptual. ● Dibuat oleh *system analyst* dan atau *system designer*.
Pemilihan DBMS	Memilih model *database management system* (DBMS)	Penentuan penggunaan *software* **SQL Server** sebagai perangkat DBMS *(database management system)*

Tahapan	Langkah	Implementasi
Logical Design	1. **Memetakan model konseptual menjadi model *logical*** Pada tahap ini, entitas dipetakan menjadi tabel-tabel. 2. **Melakukan validasi model *logical* dengan normalisasi** Normalisasi adalah proses transformasi menjadi struktur data yang lebih kecil dan stabil. Penjelasan lebih lanjut terkait normalisasi akan dijelaskan pada bagian selanjutnya. 3. **Melakukan validasi model *logical* dengan *integrity constraints*** Pada sistem, mungkin terdapat data memiliki atribut yang harus diisi dengan input tertentu. Sebagai contoh, terdapat entitas Kelas yang memiliki atribut Jam Kelas. Jam Kelas harus diisi dengan nilai di antara 7 pagi hingga 5 sore. Validasi model memastikan model memenuhi *integrity constraint* tersebut. 4. **Melakukan validasi model *logical* dengan membandingkan dengan *user requirements*** Pembuatan ***Logical Data Model (LDM)*** ● Model data yang mentranslasikan atau mendokumentasikan entitas-entitas dari model konseptual ke dalam model data yang didukung DBMS. ● Merepresentasikan cara *user* melihat data. ● Dibuat oleh *system analyst* dan/ atau *system designer.*	
Physical Design	1. **Menentukan *database* organisasi** Langkah ini mencakup menentukan data apa saja yang perlu disimpan, sehingga dapat diketahui ukuran *storage* yang diperlukan untuk *database*. Frekuensi penggunaan masing-masing *data store* juga perlu diidentifikasi. 2. **Menentukan peraturan keamanan dan integritas** Pada langkah ini, dilakukan penentuan *database role*, yaitu peranan-peranan dan keuntungan yang dimiliki oleh suatu kelompok user tertentu. Dilakukan juga penetapan pengaturan keamanan. 3. **Menentukan ukuran performansi**	Pembuatan ***Physical Data Model* (PDM)** ● Mendeskripsikan struktur fisik data. ● Merepresentasikan cara komputer menyimpan dan bekerja dengan data. ● Menampilkan atribut dari setiap entitas beserta tipe data dan ukuran *storage* yang dibutuhkan. ● Dibuat oleh *system designer.*

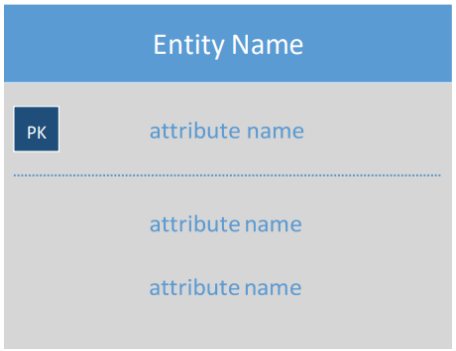
1.2 *Entity Relationship Diagram* (ERD)

Entity Relationship Diagram (ERD) merupakan salah satu model yang paling umum digunakan untuk melakukan perancangan model data konseptual. Pada modul ini, pembuatan ERD dilakukan menggunakan **Crow's Foot Database Notation** pada Microsoft Visio 2016/365.

Pada pembuatan ERD menggunakan Microsoft Visio 2016/365, terdapat sejumlah simbol yang digunakan seperti:

- **Entitas**
  Sebuah entitas dapat berupa orang, objek, event, atau konsep di dalam lingkungan pengguna di dalam organisasi yang datanya akan disimpan. Penggambaran entitas pada Microsoft Visio dapat dilihat pada Gambar 2.

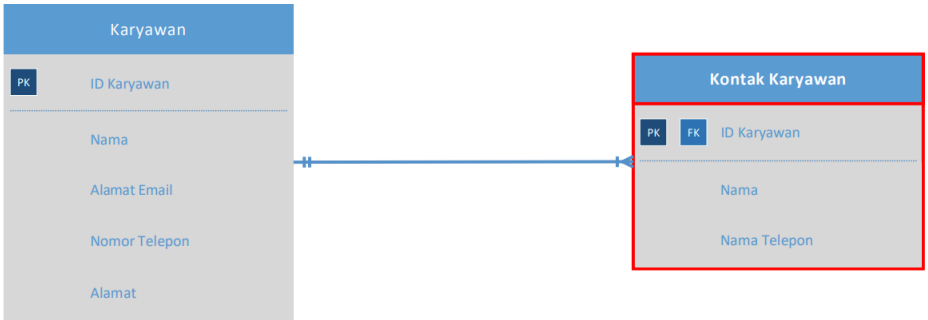
  Contoh:
  - Orang : Pegawai, mahasiswa, pasien
  - Tempat : Negara, provinsi
  - Objek : Mesin, ruangan, bangunan, pabrik
  - Event : Registrasi, penjualan, pembelian
  - Konsep : Course, account



Gambar 2 Entitas pada Microsoft Visio

- ***Child Entity***
  Child entity merupakan entitas yang memiliki karakteristik *existence dependent* dimana keberadaan (*existence*) entitas ini dependen/bergantung terhadap keberadaan entitas lain yang biasa disebut *parent entity*. *Child entity* dapat memiliki *primary key* yang sama dengan *primary key* dari *parent entity,* tetapi jika dibutuhkan dapat memiliki *primary key*-nya sendiri. Sifat *existence-dependent* menyebabkan *child entity* harus terhubung dengan *parent entity*-nya. *Child entity* memiliki simbol berupa kotak bergaris ganda.

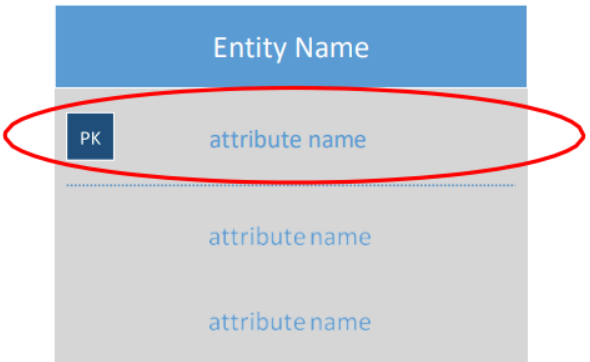
  Contoh: Terdapat entitas kontak karyawan berisi ID Karyawan, Nama, dan Nomor Telepon yang merupakan *child entity* dari entitas karyawan. Penggambaran kedua entitas ini dapat dilihat pada Gambar 3.



Gambar 3 *Parent-child entity*

- **Atribut**

  Atribut adalah properti atau karakteristik dari sebuah entitas yang berguna (untuk diketahui atau disimpan) bagi organisasi. Penempatan atribut dapat dilihat pada Gambar 4.



Gambar 4 Atribut

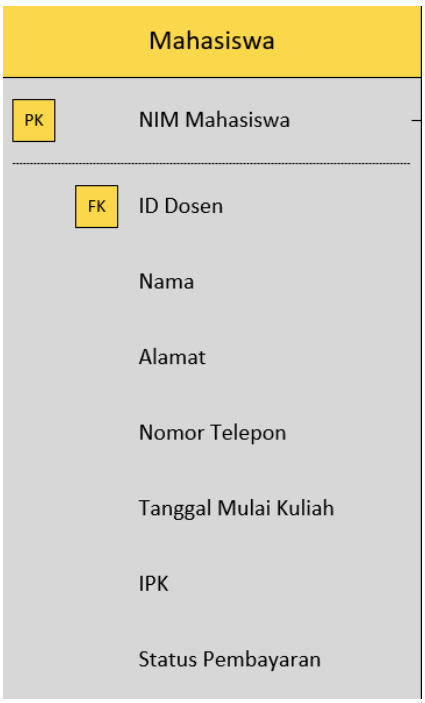
Contoh:

Entitas: Mahasiswa

Atribut:

- NIM
- Nama
- Alamat
- Nomor Telepon
- Tanggal Mulai Kuliah
- IPK
- Status Pembayaran

Gambaran dari entitas Mahasiswa beserta atribut-atributnya dapat dilihat pada Gambar 5.



Gambar 5 Entitas dan Atribut

- **Relasi**

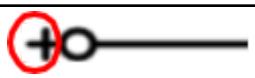
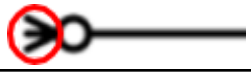
  Relasi menggambarkan asosiasi antar entitas. Jenis-jenis relasi sebagai berikut:

  a. Berdasarkan sifatnya, relasi dapat dibedakan menjadi 3, yaitu
     1. *One-to-one*
     2. *One-to-many*
     3. *Many-to-many*

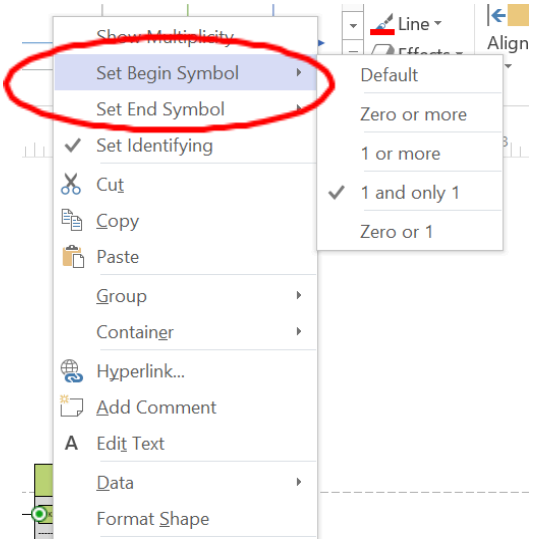
     Terdapat beberapa notasi yang dapat digunakan ketika menggambarkan relasi antar entitas. Pada praktikum ini, notasi yang akan digunakan adalah Crow's Foot Notation. Contoh dari relasi yang digambarkan menggunakan notasi Crow's Foot dapat dilihat pada Tabel 2.

     Pada relasi yang menggunakan Crow's Foot, di setiap ujung notasi terdapat dua indikator. Indikator pertama menunjukkan jumlah asosiasi maksimal entitas. Sementara itu, indikator kedua menunjukkan jumlah asosiasi minimal entitas. Berikut dijelaskan lebih lanjut mengenai simbol dan penjelasan indikator pada relasi dengan Crow's Foot.

Tabel 2 *Crow's Foot Notation*

	Jumlah asosiasi maksimal adalah satu
	Jumlah asosiasi maksimal adalah banyak (*many*)
	Jumlah asosiasi minimal adalah nol
	Jumlah asosiasi minimal adalah satu

Pada Microsoft Visio 2016/365, relasi antar entitas dapat diubah dengan melakukan klik kanan pada relasi, lalu pilih *Set Begin Symbol* dan *Set End Symbol*, lalu pilih simbol Crow's Foot yang sesuai. Simbol yang dapat dipilih berupa *zero or more*, 1 *or more*, 1 *and only* 1, dan *zero or* 1.



Gambar 6 Penetapan simbol relasi pada Microsoft Visio

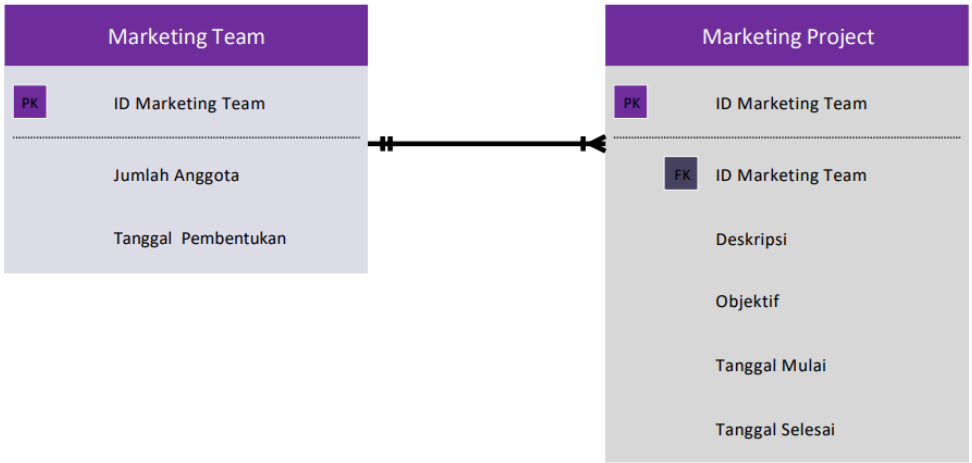
Untuk memahami lebih lanjut mengenai jenis-jenis relasi berdasarkan sifat serta perbedaannya, berikut contoh dari masing-masing jenis relasi.

- ***One-to-one***

  Pada suatu kantor terdapat entitas karyawan dan entitas komputer. Hubungan antara kedua entitas ini yaitu bahwa karyawan dapat dipinjamkan satu buah komputer atau tidak sama sekali dan satu buah komputer dapat dipinjamkan untuk satu karyawan atau tidak sama sekali. Oleh karena itu, hubungan antarentitas ini yaitu *one-on-one* dengan jumlah asosiasi maksimum keduanya, yaitu satu dan jumlah asosiasi minimumnya yaitu nol. Maka simbol yang digunakan yaitu simbol *zero or 1* pada kedua sisi.

- ***One-to-many***

  Terdapat entitas Marketing Team dan Marketing Project. Marketing Team harus melakukan satu atau lebih Marketing Project sedangkan Marketing Project hanya dapat dilakukan oleh satu Marketing Team dan pasti harus dilakukan oleh satu Marketing Team. Oleh karena itu, hubungan antarentitas ini adalah *one-to-many* dimana untuk entitas Marketing ke Marketing Project jumlah asosiasi maksimumnya adalah *many* dan minimumnya adalah satu sedangkan untuk entitas Marketing Project ke Marketing jumlah asosiasi maksimumnya adalah one dan minimumnya adalah one. Maka, simbol yang digunakan yaitu *1 and only 1* (dari Marketing Project ke Marketing Team) dan *1 or more* (dari Marketing Team ke Marketing Project). Penggambaran dari entitas Marketing Team dan Marketing Project beserta relasinya dapat dilihat pada Gambar 7.



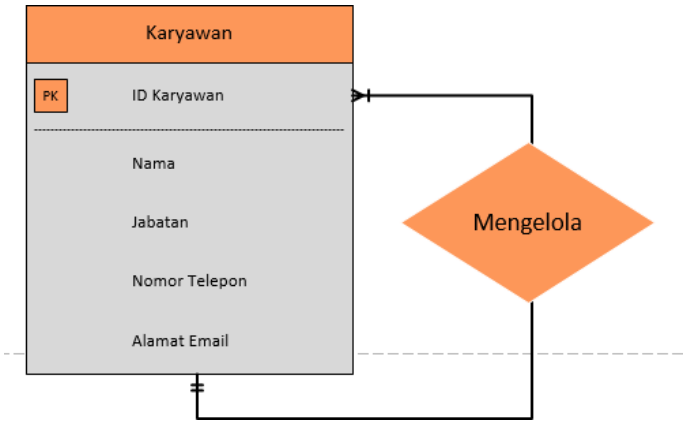
Gambar 7 Contoh relasi *one-to-many* antarentitas

- ***Many-to-many***

  Terdapat entitas Karyawan dan Project. Karyawan dapat memilih untuk melakukan projek (baik 1 maupun lebih) atau tidak sama sekali sedangkan Project harus dikerjakan oleh minimal satu karyawan. Oleh karena itu, kedua entitas ini memiliki hubungan *many-to-many*, dimana untuk entitas Karyawan ke Project jumlah asosiasi minimum nya yaitu nol dan jumlah asosiasi maksimumnya yaitu banyak (*many*) sedangkan untuk Project ke Karyawan jumlah asosiasi minimumnya yaitu satu dan jumlah asosiasi maksimumnya yaitu banyak (*many*). Maka simbol yang digunakan adalah *zero or more* (Karyawan ke Project) dan *1 or more* (Project ke Karyawan).

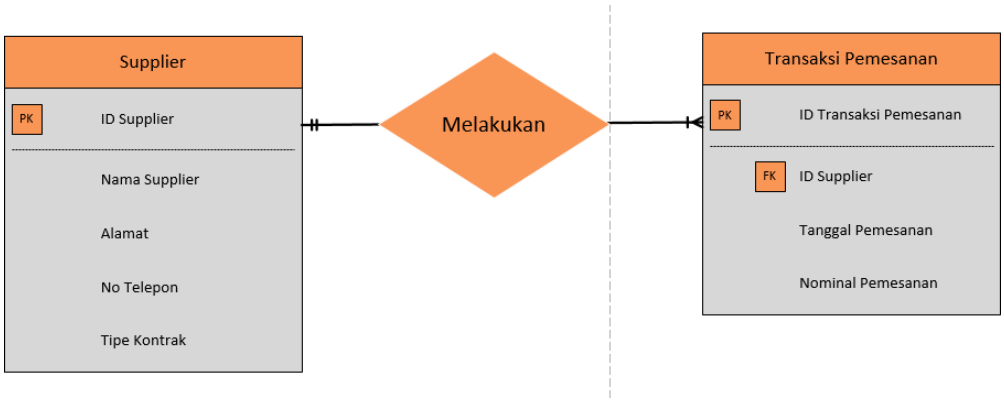
b. Berdasarkan tingkatannya, relasi dapat dibagi menjadi 3, yaitu:

- ***Unary relationship***
  Unary relationship adalah relasi antara satu entitas.



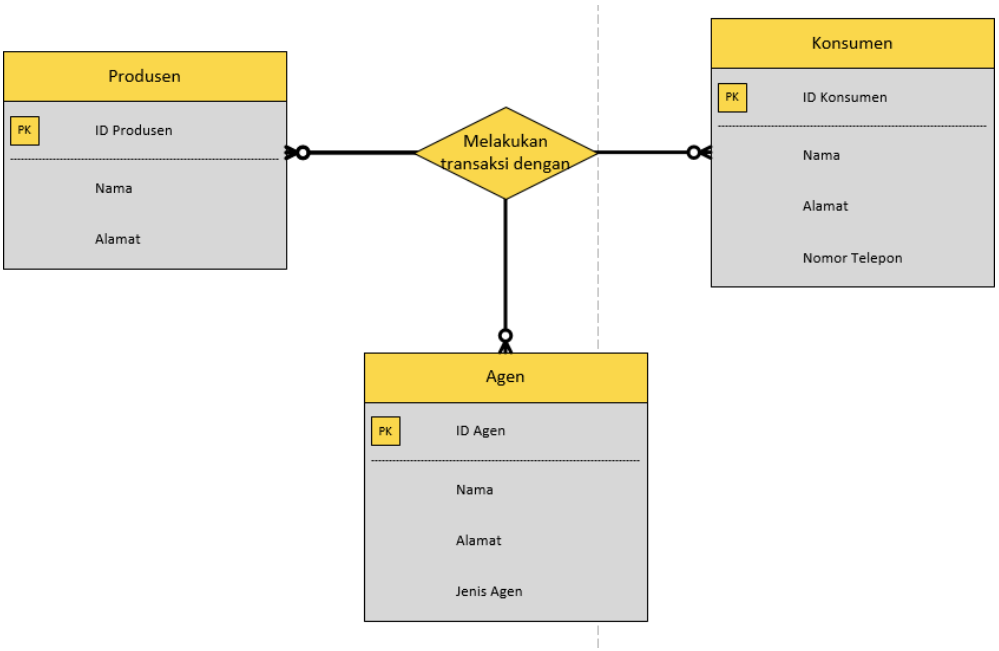
Gambar 8 *Unary relationship*

- ***Binary relationship***
  Binary relationship adalah relasi antara dua entitas.



Gambar 9 *Binary relationship*

- ***Ternary relationship***
  Ternary relationship adalah relasi antara lebih dari dua entitas.

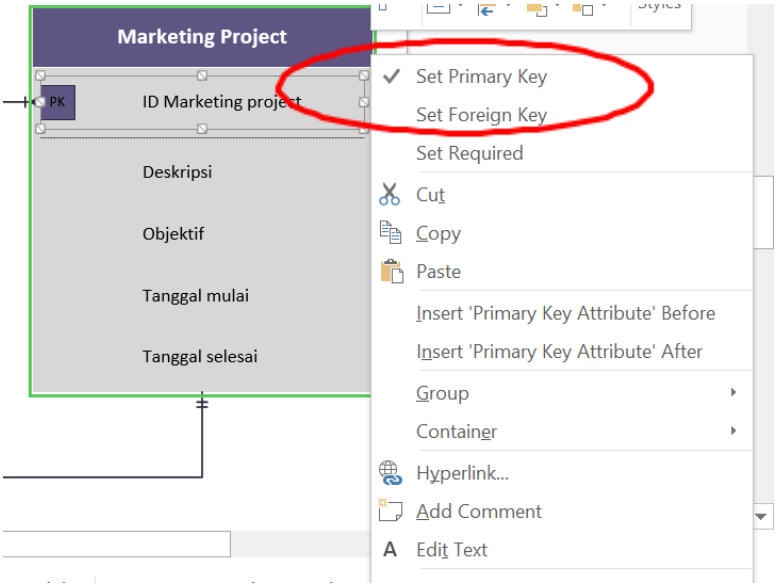


Gambar 10 *Ternary relationship*

Ketika membentuk suatu relasi dalam pemodelan data, maka terdapat atribut-atribut yang menjadi acuan dalam menghubungkan entitas-entitas yang ada. Atribut-atribut ini disebut sebagai *primary key* dan *foreign key*. Berikut merupakan penjelasan mengenai *primary key* dan *foreign key*.

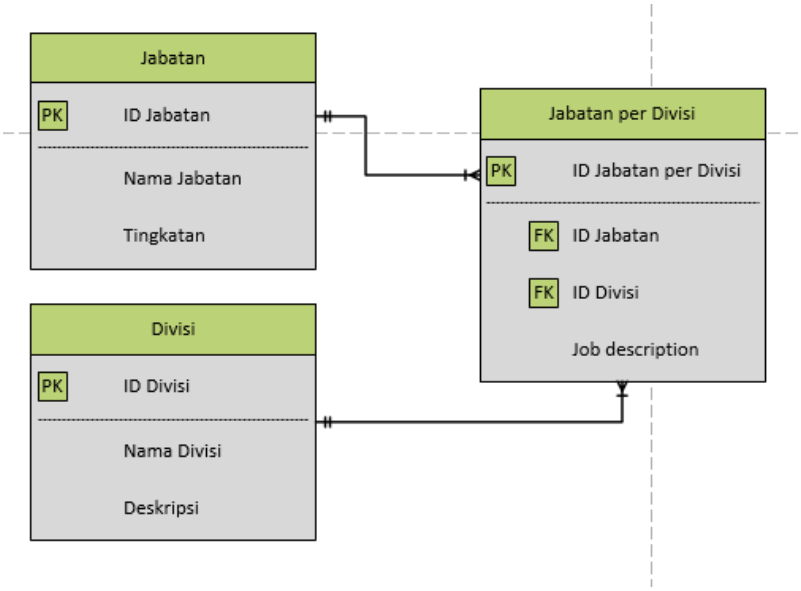
- ***Primary Key***
  Primary key merupakan suatu nilai dalam basis data yang digunakan untuk mengidentifikasi suatu baris dalam tabel. Nilai ini bersifat unik untuk setiap *record* dan umumnya digunakan sebagai identitas suatu entitas. Hal ini berarti Anda tidak dapat menambahkan 2 record dengan *primary key* yang sama.
- ***Foreign Key***
  Foreign key merupakan suatu atribut yang berperan sebagai penghubung antar entitas yang berkaitan. *Foreign key* melengkapi hubungan suatu entitas terhadap *primary key* yang berkaitan dengan entitas induknya. Jika sebuah *primary key* terhubung dengan entitas lainnya, maka keberadaan *primary key* pada entitas tersebut disebut sebagai *foreign key*.

Pada Microsoft Visio, penetapan suatu atribut sebagai *primary key* dan *foreign key* dilakukan dengan melakukan klik kanan pada atribut tabel entitas, lalu memilih “Set Primary Key” atau “Set Foreign Key”.



Gambar 11 Pengaturan *primary key* dan *foreign key* pada Microsoft Visio

Berikut ini merupakan contoh penggunaan *primary key* dan *foreign key* pada ERD.



Gambar 12 Contoh penggunaan *primary key* dan *foreign key*

1.3 Normalisasi

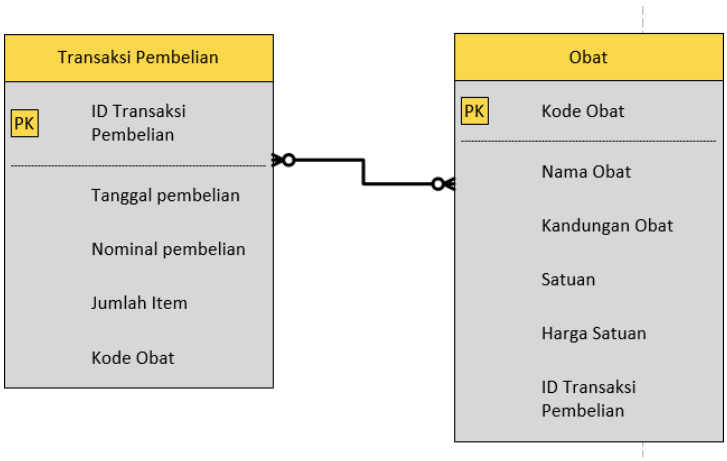
Normalisasi merupakan teknik yang digunakan untuk memastikan sebuah *database* terbebas dari redundansi data. Redundansi data yaitu suatu kondisi ketika terdapat sejumlah informasi sama yang tersimpan di dalam *database*. Redundansi data dapat menimbulkan banyak permasalahan. Berikut merupakan permasalahan yang dapat disebabkan oleh redundansi data.

- *Data inconsistency (lack of data integrity)*
- *Data anomalies*
  Data anomalies dapat dibedakan menjadi tiga, yaitu:
  - *Modification anomalies*: suatu kondisi dimana data tidak konsisten terjadi karena adanya modifikasi
  - *Insertion anomalies*: suatu kondisi dimana data tidak konsisten terjadi karena adanya penambahan data
  - *Deletion anomalies*: suatu kondisi dimana data tidak konsisten terjadi karena adanya penghapusan data

Umumnya, normalisasi dilakukan untuk entitas-entitas yang memiliki relasi *many-to-many* karena relasi ini banyak menimbulkan redundansi pada entitas-entitas yang bersangkutan.

Contoh Kasus Normalisasi

Diberikan dua buah entitas yaitu Transaksi Pembelian dan Obat. Kedua entitas ini memiliki hubungan *many-to-many* karena satu obat dapat dibeli dalam banyak pembelian dan pada satu transaksi pembelian dapat terbeli banyak obat. Hubungan ini dapat dilihat pada gambar di bawah ini:



Gambar 13 Relasi antara entitas Obat dan Transaksi Pembelian

Jika dilihat dari Tabel 3 dan Tabel 4, terdapat dua *record* yang memiliki informasi yang sama atau disebut juga dengan redundansi data.

Tabel 3 Transaksi Pembelian

ID Transaksi Pembelian	Tanggal Pembelian	Nominal Pembelian	Kode Obat	Jumlah Item
321	4 Maret 2025	Rp14.000	B001	1
324	12 Maret 2025	Rp6.000	B002	1
342	14 Maret 2025	Rp25.000	B003	1
349	21 Maret 2025	Rp135.000	B003	2
349	21 Maret 2025	Rp135.000	B004	1

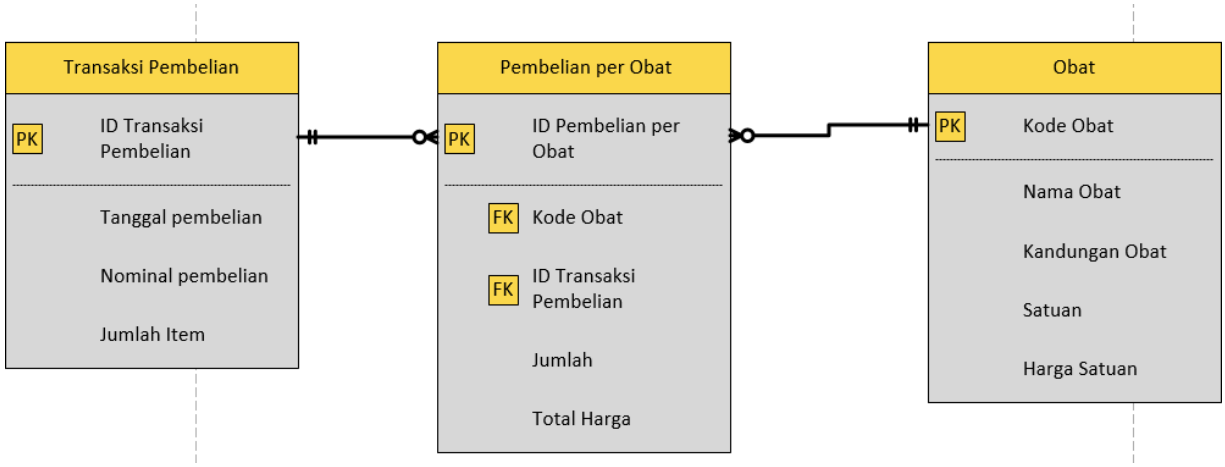
Tabel 4 Obat

Kode Obat	Nama Obat	Kandungan Obat	Satuan	Harga Satuan	ID Transaksi Pembelian
B001	Panadol	Parasetamol	Strip	Rp14.000	321
B002	Antimo	Dimenhidrinat	Strip	Rp6.000	324
B003	OBH Combi Plus	Succus liquiritiae, Ammonium chloride	Botol	Rp25.000	342
B003	OBH Combi Plus	Succus liquiritiae, Ammonium chloride	Botol	Rp25.000	349
B004	Mastin	Antosianin	Box	Rp85.000	349

Redundansi data seperti yang terlihat pada Tabel 3 dan Tabel 4 tidak efisien dengan alasan sebagai berikut:

- Menyebabkan pemborosan pada memori.
- Menyebabkan kesulitan untuk melakukan *update* data pada setiap tabelnya karena harus dikerjakan dua kali pada tabel yang berbeda.
- Peluang kesalahan pada data semakin besar jika hal ini terjadi (contoh: bisa jadi saat *update* lupa untuk melakukan *update* di salah satu tabel sehingga terjadi *discrepancy*).

Untuk menghindari hal tersebut, maka dilakukanlah metode normalisasi dengan cara membuat entitas baru di antara dua entitas yang memiliki hubungan *many-to-many* seperti gambar di bawah ini:



Gambar 14 Hasil Normalisasi

Entitas hasil normalisasi akan memiliki hubungan *many-to-one* dengan entitas-entitas asalnya. Pada kasus Transaksi dan Obat, hasil tabel normalisasi yaitu Pembelian per Obat akan memiliki hubungan *many-to-one* dengan Transaksi Pembelian dan Obat. Hal ini dikarenakan satu pembelian per obat hanya terdapat di satu transaksi dan hanya meliputi satu jenis obat saja. Berikut merupakan tabel hasil normalisasi yang telah dibuat.

Tabel 5 Transaksi Pembelian Hasil Normalisasi

ID Transaksi Pembelian	Tanggal Pembelian	Nominal Pembelian	Jumlah Item
321	4 Maret 2025	Rp14.000	1
324	12 Maret 2025	Rp6.000	1
342	14 Maret 2025	Rp25.000	1
349	21 Maret 2025	Rp135.000	3

Tabel 6 Obat Hasil Normalisasi

Kode Obat	Nama Obat	Kandungan Obat	Satuan	Harga Satuan
B001	Panadol	Parasetamol	Strip	Rp14.000
B002	Antimo	Dimenhidrinat	Strip	Rp6.000
B003	OBH Combi Plus	Succus liquiritiae, Ammonium chloride	Botol	Rp25.000
B004	Mastin	Antosianin	Box	Rp85.000

Tabel 7 Pembelian per Obat Hasil Normalisasi

ID Pembelian per Obat	ID Transaksi Pembelian	Kode Obat	Jumlah	Total Harga
P001	321	B001	1	Rp14.000
P002	324	B002	1	Rp6.000
P003	342	B003	1	Rp25.000
P004	349	B003	2	Rp50.000
P005	349	B004	1	Rp85.000

II. STUDI KASUS PEMODELAN DATA

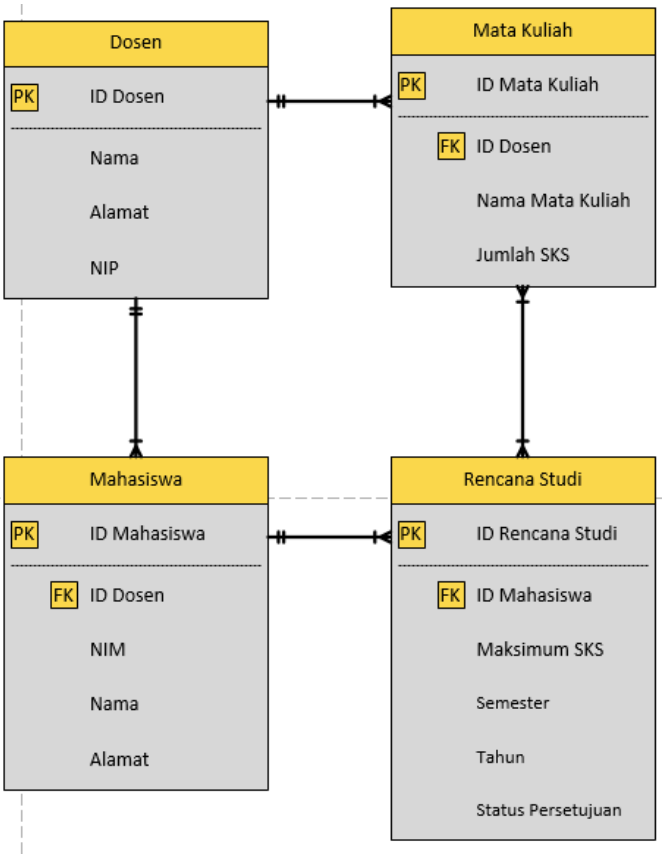
Sebuah sekolah tinggi yang baru saja berdiri ingin mengimplementasikan sebuah sistem informasi untuk memudahkan para mahasiswanya dalam melakukan pengisian rencana studi. Dengan sistem informasi yang berbasis web ini, mahasiswa sekolah tinggi tersebut dapat memilih berbagai macam mata kuliah yang tersedia sesuai dengan ketentuan yang berlaku. Setelah mahasiswa mengambil suatu mata kuliah, sistem akan menghubungkan mata kuliah yang diambil tersebut dengan dosen pengajar dan ruangan kelasnya. Namun karena sekolah tinggi yang baru saja didirikan ini masih mengalami sejumlah masalah internal, terdapat sejumlah ketentuan yang perlu dipertimbangkan dalam pembuatan sistem. Adapun ketentuan tersebut adalah sebagai berikut.

- Setiap mahasiswa akan memiliki dosen wali. Tugas dosen wali ini adalah melakukan verifikasi atas rencana studi yang dipilih oleh mahasiswa dengan ketentuan sekolah tinggi.
- Karena keterbatasan jumlah dosen, seorang dosen dapat mengajar sejumlah mata kuliah namun setiap mata kuliah hanya akan diampu oleh satu dosen.

Berikut adalah contoh hasil pembuatan ERD, LDM, dan PDM berdasarkan studi kasus.

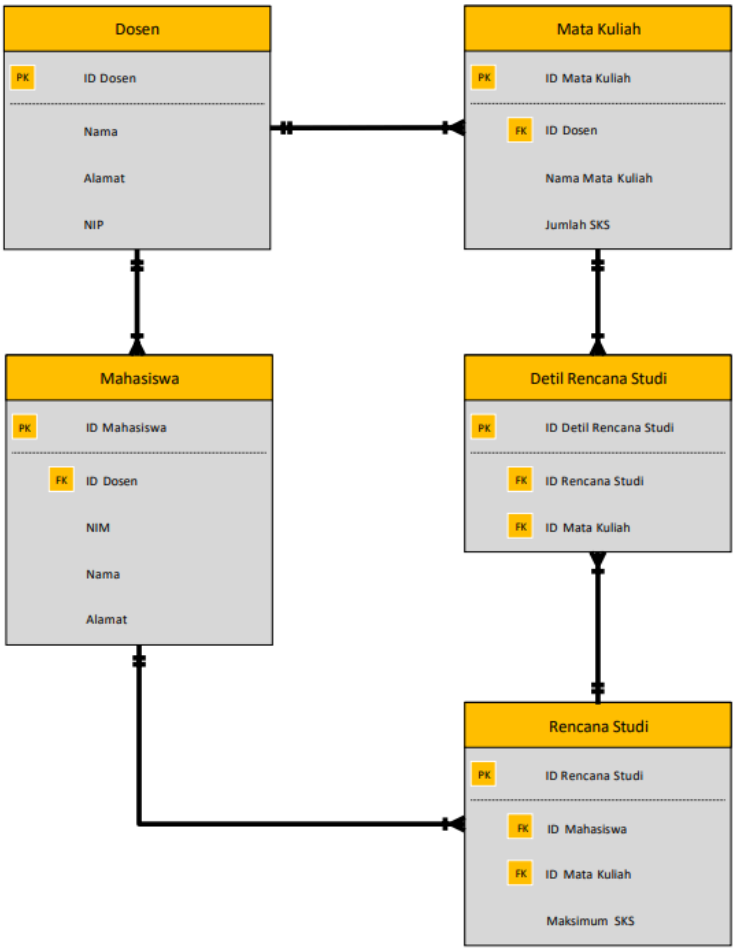
2.1 ERD

Berikut merupakan ilustrasi ERD sederhana untuk pengisian rencana studi *online*



Gambar 15 ERD Pengisian Rencana Studi

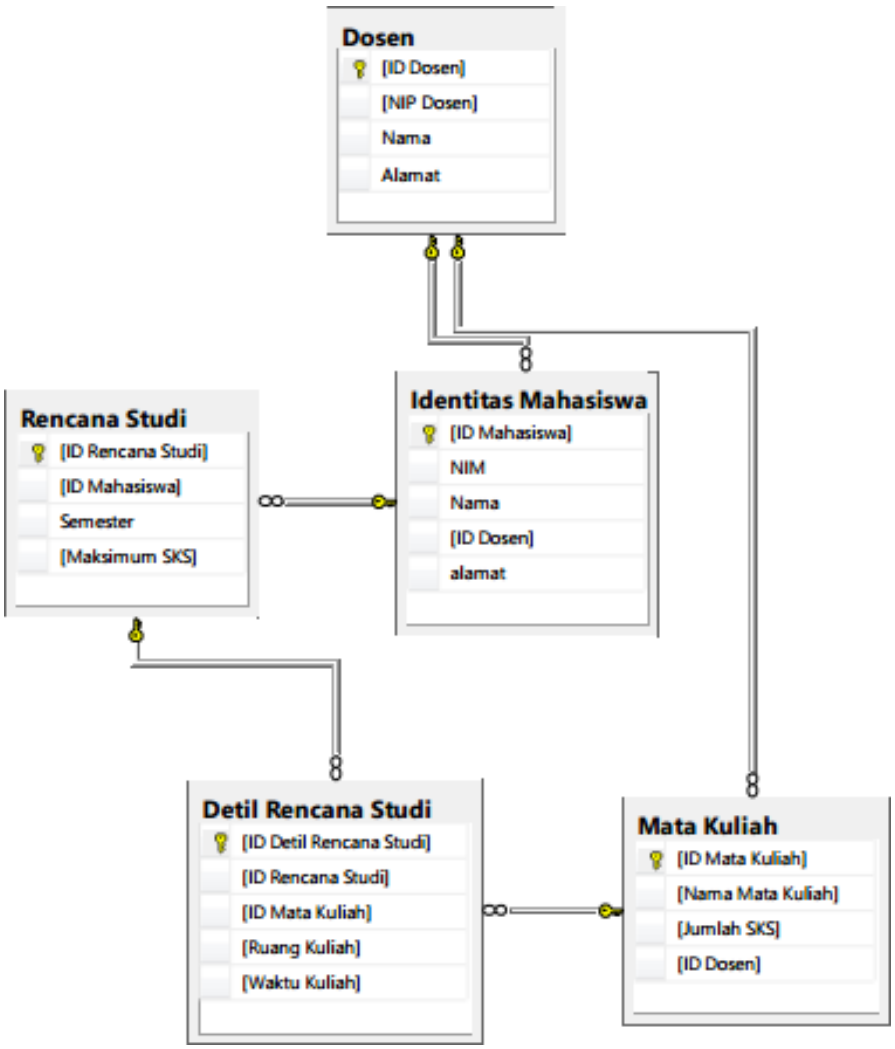
Pada gambar di atas dapat dilihat adanya hubungan *many to many* antara entitas Rencana Studi dan Mata Kuliah sehingga perlu dilakukan normalisasi. Berikut adalah hasil normalisasi yang dilakukan.



Gambar 16 Normalisasi ERD Pengisian Rencana Studi

2.2 LDM

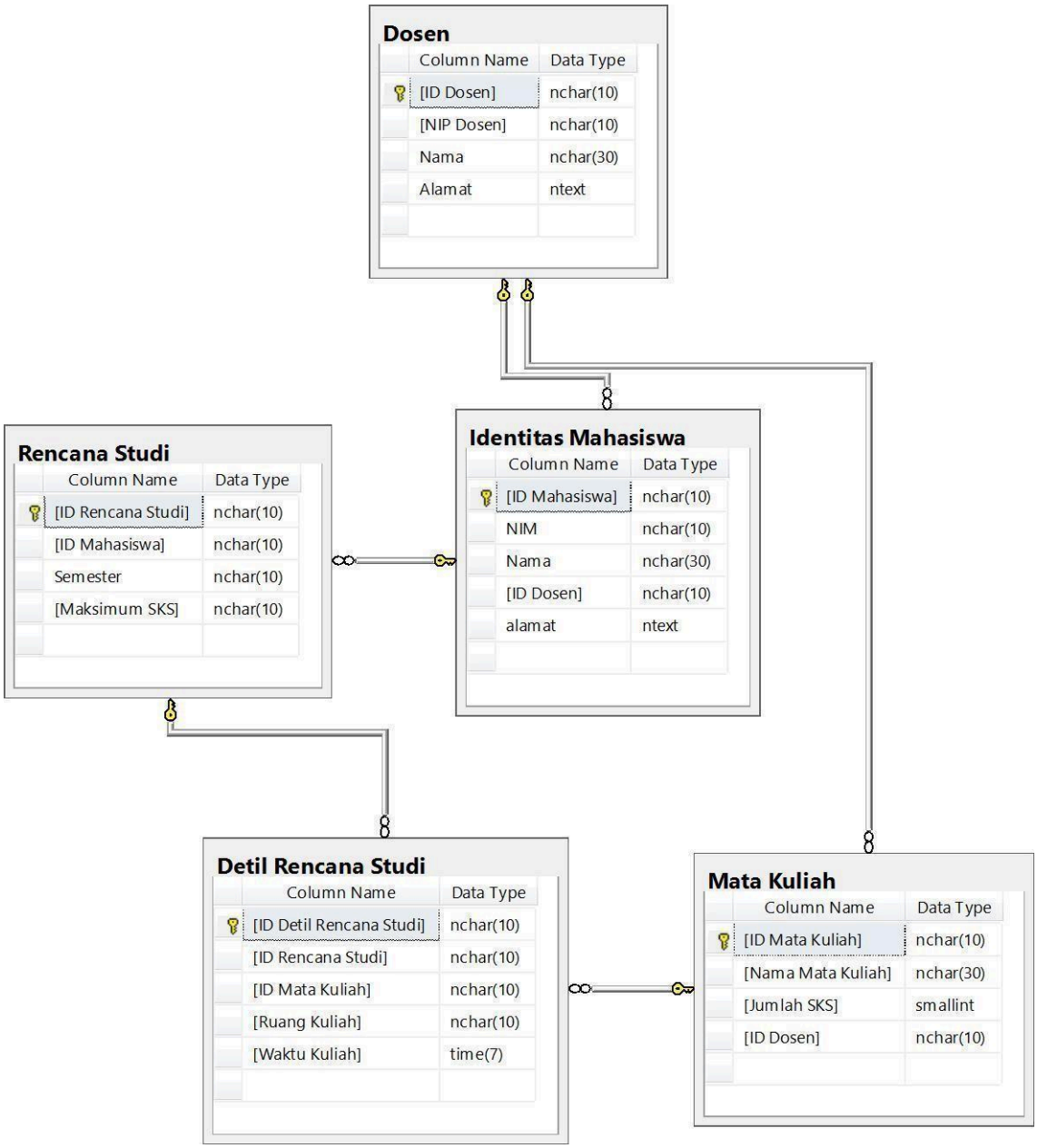
Berikut merupakan LDM yang diperoleh dari Microsoft SQL Server 2012.



Gambar 17 LDM Pengisian Rencana Studi

2.3 PDM

Berikut merupakan PDM yang diperoleh dari Microsoft SQL Server 2012.



Gambar 18 PDM Pengisian Rencana Studi

REFERENSI

Coronel, C., Morris, S., & Rob, P. (2011). *Database Systems: Design, Implementation, and Management 9th Edition.* Boston: Cengage Learning.

Whitten, J. L., & Bentley, L. D. (2007). *Systems Analysis and Design Methods 7th Edition.* New York: McGraw-Hill.